

app\api_client.py

```

1  # -*- coding: utf-8 -*-
2  """
3  @file      api_client.py
4  @brief     Client API pour les services oblosolutions.ch.
5  @details   Ce module fournit un client HTTP pour interroger les APIs
6             td25_param et td25_forecast. Il gère les timeouts, les retries
7             automatiques et les erreurs de réseau. Inclut des fonctions
8             utilitaires pour récupérer les données de simulation et envoyer
9             les mesures de température.
10 @author    Léo Mendes
11 @project   2510_RegulationsChauffage
12 @Mandant   Oblo_solution
13 @date      2025-09-18
14 @version   1.0.0
15 @copyright Copyright (c) 2025
16 """
17
18 from __future__ import annotations
19
20 # Client HTTP pour les requêtes REST
21 import requests
22 # Fonctions temporelles (délais, sleep)
23 import time
24 # Sérialisation/désérialisation JSON
25 import json
26 # Types pour l'annotation de code
27 from typing import Dict, Optional, Any
28
29 # -----
30 # Constantes du module
31 # -----
32
33 # Délai d'attente pour les requêtes HTTP [s]
34 REQUEST_TIMEOUT = 10
35 # Nombre maximum de tentatives en cas d'échec
36 MAX_RETRIES = 3
37 # Délai entre les tentatives [s]
38 RETRY_DELAY = 1
39
40 # -----
41 # Classes d'exception
42 # -----
43
44 class ApiError(Exception):
45     """
46     @brief   Exception pour les erreurs d'API.
47     @details Exception personnalisée levée en cas d'erreur lors des
48             communications avec les APIs oblosolutions.ch.
49     """
50     pass
51
52 # -----
53 # Classes principales
54 # -----
55
56 class ApiClient:
57     """
58     @brief   Client pour interroger les APIs oblosolutions.ch.
59     @details Fournit des méthodes pour récupérer les paramètres de simulation,
60             les prévisions météo et envoyer les mesures de température.
61             Gère automatiquement les retries et les timeouts.
62     """
63
64     # Pas de préfixe, endpoints à la racine
65     API_PATH_PREFIX = ""
66
67     def __init__(self, mac_address: str):
68         """
69         @brief   Initialise le client API.
70         @details Configure l'adresse MAC du dispositif et l'URL de base
71                 pour les communications avec les services oblosolutions.ch.
72
73         @param   mac_address Adresse MAC du dispositif (ex: "0030DEABCDE").
74         """
75         # Stockage de l'adresse MAC du dispositif
76         self.mac_address = mac_address
77         # URL de base pour tous les appels API
78         self.base_url = "https://dev.oblosolutions.ch"
79
80     def get_parameters(self) -> Dict[str, Any]:
81         """
82         @brief   Récupère les paramètres depuis l'API td25_param.
83         @details Effectue une requête GET vers l'endpoint td25_param avec
84                 l'adresse MAC comme paramètre. Parse la réponse JSON et

```

```

85         extrait les champs probe_type, n, k_m et temperature.
86         Inclut un système de retry automatique en cas d'échec.
87
88     @return Dictionnaire contenant probe_type, n, k_m, temperature.
89
90     @exception ApiError En cas d'erreur de récupération des données.
91     """
92     # Construction de l'URL pour l'endpoint des paramètres
93     url = f"{self.base_url}/td25_param"
94     # Paramètres de la requête GET
95     params = {"mac_address": self.mac_address}
96     # Affichage de la requête pour le debug
97     print(f"[API] GET {url} params={params}")
98
99     # Boucle de retry en cas d'échec
100    for attempt in range(MAX_RETRIES):
101        try:
102            # Exécution de la requête GET avec timeout
103            response = requests.get(url, params=params, timeout=REQUEST_TIMEOUT)
104
105            # Vérification du code de statut HTTP
106            if response.status_code != 200:
107                # Lever une exception en cas d'erreur HTTP
108                raise ApiError(f"Erreur HTTP {response.status_code}: {response.text}")
109
110            # Parsing de la réponse JSON
111            data = response.json()
112            # Vérification de la présence du champ 'param'
113            if "param" not in data or not data["param"]:
114                raise ApiError("Champ 'param' manquant ou vide")
115            # Extraction du premier élément des paramètres
116            entry = data["param"][0]
117            # Extraction des champs individuels
118            probe_type = entry.get("probe_type")
119            n = entry.get("n")
120            k_m = entry.get("k_m")
121            temperature = entry.get("temperature")
122            # Vérification que tous les champs requis sont présents
123            if None in (probe_type, n, k_m, temperature):
124                raise ApiError("Un ou plusieurs champs requis sont manquants dans la réponse API")
125            # Affichage des paramètres récupérés
126            print(f"[API] Paramètres récupérés: {entry}")
127            # Retour du dictionnaire formaté
128            return {
129                "probe_type": probe_type,
130                "n": n,
131                "k_m": k_m,
132                "temperature": temperature
133            }
134
135        except requests.exceptions.RequestException as e:
136            # Gestion des erreurs de réseau
137            print(f"[API] Tentative {attempt + 1}/{MAX_RETRIES} échouée: {e}")
138            # Attente avant retry si ce n'est pas la dernière tentative
139            if attempt < MAX_RETRIES - 1:
140                time.sleep(RETRY_DELAY)
141            else:
142                # Lever une exception après épuisement des tentatives
143                raise ApiError(f"Impossible de récupérer les paramètres après {MAX_RETRIES} tentatives: {e}")
144        except json.JSONDecodeError as e:
145            # Gestion des erreurs de parsing JSON
146            raise ApiError(f"Réponse JSON invalide: {e}")
147
148    def get_forecast(self) -> Dict[str, Any]:
149        """
150        @brief Récupère la température prévue depuis l'API td25_forecast.
151        @details Effectue une requête GET vers l'endpoint td25_forecast avec
152                 l'adresse MAC comme paramètre. Parse la réponse JSON et
153                 extrait la température de prévision météorologique.
154                 Inclut un système de retry automatique en cas d'échec.
155
156        @return Dictionnaire contenant la temperature prévue.
157
158        @exception ApiError En cas d'erreur de récupération des données.
159        """
160        # Construction de l'URL pour l'endpoint des prévisions
161        url = f"{self.base_url}/td25_forecast"
162        # Paramètres de la requête GET
163        params = {"mac_address": self.mac_address}
164        # Affichage de la requête pour le debug
165        print(f"[API] GET {url} params={params}")
166
167        # Boucle de retry en cas d'échec
168        for attempt in range(MAX_RETRIES):
169            try:
170                # Exécution de la requête GET avec timeout
171                response = requests.get(url, params=params, timeout=REQUEST_TIMEOUT)

```

```

172
173     # Vérification du code de statut HTTP
174     if response.status_code != 200:
175         # Lever une exception en cas d'erreur HTTP
176         raise ApiError(f"Erreur HTTP {response.status_code}: {response.text}")
177
178     # Parsing de la réponse JSON
179     data = response.json()
180     # Vérification de la présence du champ 'forecast'
181     if "forecast" not in data or not data["forecast"]:
182         raise ApiError("Champ 'forecast' manquant ou vide")
183     # Extraction du premier élément des prévisions
184     forecast = data["forecast"][0]
185     # Extraction de la température de prévision
186     temperature_forecast = forecast["temperature"]
187     # Affichage des prévisions récupérées
188     print(f"[API] Prévisions récupérées: {forecast}")
189     # Retour du dictionnaire avec la température
190     return {"temperature": temperature_forecast}
191
192 except requests.exceptions.RequestException as e:
193     # Gestion des erreurs de réseau
194     print(f"[API] Tentative {attempt + 1}/{MAX_RETRIES} échouée: {e}")
195     # Attente avant retry si ce n'est pas la dernière tentative
196     if attempt < MAX_RETRIES - 1:
197         time.sleep(RETRY_DELAY)
198     else:
199         # Lever une exception après épuisement des tentatives
200         raise ApiError(f"Impossible de récupérer les prévisions après {MAX_RETRIES} tentatives: {e}")
201 except json.JSONDecodeError as e:
202     # Gestion des erreurs de parsing JSON
203     raise ApiError(f"Réponse JSON invalide: {e}")
204
205 def post_measured_temperature(self, temperature: float, channel: int = None) -> bool:
206     """
207     @brief  Envoie la température mesurée vers l'API td25_param.
208     @details Effectue une requête POST vers l'endpoint td25_param avec
209             la température mesurée et optionnellement le numéro de canal.
210             Tous les paramètres sont envoyés dans l'URL comme paramètres
211             de requête. Inclut un système de retry automatique.
212
213     @param  temperature Température mesurée en °C.
214     @param  channel      Canal de mesure (optionnel).
215
216     @return  True si succès, False sinon.
217     """
218     # Construction de l'URL pour l'endpoint des paramètres
219     url = f"{self.base_url}/td25_param"
220
221     # Préparation des paramètres URL (comme dans l'exemple fourni)
222     # Paramètres de base pour l'envoi
223     params = {
224         "mac_address": self.mac_address,
225         "measured_temp": temperature
226     }
227
228     # Ajout du canal si spécifié
229     if channel is not None:
230         params["channel"] = channel
231
232     # Affichage de la requête pour le debug
233     print(f"[API] POST {url} params={params}")
234
235     # Boucle de retry en cas d'échec
236     for attempt in range(MAX_RETRIES):
237         try:
238             # Exécution de la requête POST avec paramètres en URL
239             response = requests.post(
240                 url,
241                 # Tout en paramètres URL
242                 params=params,
243                 timeout=REQUEST_TIMEOUT
244             )
245
246             # Vérification du succès de la requête
247             if response.status_code in [200, 201]:
248                 print(f"[API] Température envoyée avec succès: {temperature}°C")
249                 return True
250             else:
251                 # Affichage de l'erreur HTTP
252                 print(f"[API] Erreur HTTP {response.status_code}: {response.text}")
253                 # Retry si ce n'est pas la dernière tentative
254                 if attempt < MAX_RETRIES - 1:
255                     time.sleep(RETRY_DELAY)
256                 continue
257             else:
258                 return False

```

```

259
260     except requests.exceptions.RequestException as e:
261         # Gestion des erreurs de réseau
262         print(f"[API] Tentative {attemp + 1}/{MAX_RETRIES} d'envoi échouée: {e}")
263         # Attente avant retry si ce n'est pas la dernière tentative
264         if attemp < MAX_RETRIES - 1:
265             time.sleep(RETRY_DELAY)
266         else:
267             print(f"[API] Impossible d'envoyer la température après {MAX_RETRIES} tentatives")
268             return False
269
270     return False
271
272 # -----
273 # Fonctions utilitaires
274 # -----
275
276 def get_simulation_data(mac_address: str) -> Dict[str, Any]:
277     """
278     @brief Récupère les données de simulation depuis les APIs.
279     @details Combine les appels vers td25_param et td25_forecast pour
280             récupérer tous les paramètres nécessaires à la simulation
281             de température. Utilise le client ApiClient pour gérer
282             les retries et les erreurs automatiquement.
283
284     @param mac_address Adresse MAC du dispositif.
285
286     @return Dictionnaire avec toutes les données nécessaires pour la simulation.
287
288     @exception ApiError Si aucune donnée valide n'est obtenue.
289     """
290     # Création d'une instance du client API
291     client = ApiClient(mac_address)
292
293     # Récupération automatique
294     print("[SIMULATION] Tentative de récupération automatique des données...")
295
296     # Récupération des paramètres
297     params = client.get_parameters()
298
299     # Récupération des prévisions
300     forecast = client.get_forecast()
301
302     # Combinaison des données
303     # Création du dictionnaire de données de simulation
304     simulation_data = {
305         "probe_type": params["probe_type"],
306         "n": float(params["n"]),
307         "k_m": float(params["k_m"]),
308         "temperature": float(params["temperature"]),
309         "forecast_temperature": float(forecast["temperature"])
310     }
311
312     # Confirmation du succès de la récupération
313     print("[SIMULATION] Données récupérées automatiquement avec succès")
314     return simulation_data
315
316 def send_temperature_measurement(mac_address: str, temperature: float, channel: int = None) -> bool:
317     """
318     @brief Envoie une mesure de température vers l'API.
319     @details Fonction utilitaire qui crée un client ApiClient et utilise
320             la méthode post_measured_temperature pour envoyer la température
321             mesurée. Gère les exceptions et retourne un booléen simple.
322
323     @param mac_address Adresse MAC du dispositif.
324     @param temperature Température mesurée en °C.
325     @param channel Canal de mesure (optionnel).
326
327     @return True si succès, False sinon.
328     """
329     # Création d'une instance du client API
330     client = ApiClient(mac_address)
331
332     try:
333         # Tentative d'envoi de la température
334         return client.post_measured_temperature(temperature, channel)
335     except Exception as e:
336         # Gestion des erreurs générales
337         print(f"[API] Erreur lors de l'envoi de la température: {e}")
338         return False
339
340 # -----
341 # Point d'entrée pour les tests
342 # -----
343
344 # Test du module (à des fins de développement)
345 if __name__ == "__main__":

```

```
346 # Adresse MAC de test
347 test_mac = "0030DEABCDEF"
348
349 try:
350     # Tentative de récupération des données de simulation
351     data = get_simulation_data(test_mac)
352     # Affichage des données récupérées
353     print(f"\nDonnées finales: {data}")
354 except ApiError as e:
355     # Gestion des erreurs spécifiques à l'API
356     print(f"Erreur: {e}")
```